Trabajo práctico
Nave Interestelar

Docentes a cargo: Leonel Domingo Guccione, Lucas de Lellis y Micaela Rasso.

Estudiantes: Jeremías Joaquín Bernardi, Nestor Uriel Aristegui, Santiago Martinez y Agustin Veca.

Asignatura: Programación C.

Carrera: Ingeniería Informática.

Facultad de Ingeniería- UNMDP.

Descripción General del Proyecto:

El proyecto consiste en el desarrollo del "Software embarcado de una nave interestelar tripulada", correspondiente al Trabajo Práctico Grupal (TPG) de la asignatura Programación C del año 2026. Toda la documentación de referencia para el desarrollo de la solución puede vincularse al archivo.

Objetivo del Sistema

El objetivo principal es construir y hacer evolucionar un sistema orientado a objetos desarrollado en Java y estructurado con Maven. Este software funcionará como un asistente de navegación y comando para la nave. Sus responsabilidades incluyen administrar recursos limitados (como combustible y energía), coordinar subsistemas (como el Motor Warp), ejecutar misiones y mantener registros detallados de los eventos en una bitácora, todo operando bajo las órdenes y autoridad del capitán.

Estructura y Evolución del Trabajo

El desarrollo está planteado en dos etapas incrementales:

- **Entrega 1 (E1):** Consiste en la construcción del núcleo orientado a objetos en un entorno controlado. En esta fase se desarrollan los modelos de la nave (Exploradora, Carguero, Combate), la tripulación, el cálculo de haberes, y versiones simplificadas de tres misiones (Intercepción, Recolección y Retorno seguro). El núcleo debe ser independiente de consolas de texto y de interfaces gráficas.
- **Entrega 2 (E2):** El núcleo previamente construido se integrará con un Motor de Simulación provisto por la cátedra para operar en un universo dinámico sobre un tablero de 20x20. En esta etapa se agregarán características avanzadas como sensores, reglas de navegación, persistencia de datos mediante JDBC, concurrencia y una interfaz gráfica basada en el patrón MVC.

Arquitectura y Patrones de Diseño Obligatorios

Para la primera entrega, el sistema exige establecer una arquitectura sólida mediante la implementación de cuatro patrones de diseño específicos:

- **Factory:** Utilizado para delegar la creación y configuración inicial de los tres tipos de naves sin acoplar el código cliente a las clases concretas.
- **State:** Empleado para modelar el ciclo de vida del Motor Warp, asegurando que solo se permitan transiciones lógicas entre los estados de disponibilidad, preparación, salto y enfriamiento.
- **Template Method:** Aplicado para establecer un ciclo general inalterable en las misiones con cuatro etapas (preparar, ejecutar, evaluar y cerrar), permitiendo que cada misión específica redefina solo lo necesario.
- **Decorator:** Usado para calcular dinámicamente la remuneración de la tripulación, componiendo el sueldo base del cargo con adicionales por antigüedad, origen planetario y plus de consejos, evitando así una jerarquía rígida de clases.

Organización del Equipo

El proyecto es desarrollado de manera colaborativa a través de un repositorio en GitHub por un equipo conformado por cuatro integrantes: Santiago, Uriel, Agustin y Jeremías. Para la Entrega 1, el grupo dividió equitativamente los 10 requerimientos obligatorios (E1-01 a E1-10) asignando a cada miembro componentes específicos del dominio y la implementación de un patrón de diseño particular para nivelar la carga de trabajo.

Justificación y Aplicación de los Patrones de Diseño Obligatorios

El núcleo orientado a objetos requería la implementación de cuatro patrones de diseño específicos para resolver problemas concretos del dominio.

- **Patrón Factory (Creación de naves - E1-07):**
  - Este requerimiento fue asignado a Santiago, responsable también del modelo de la nave y la gestión de recursos.
  - El patrón se aplica para instanciar los tres tipos de nave exigidos: Exploradora, Carguero y Combate.
  - **Justificación:** Se utiliza para que el código cliente pueda solicitar una nave sin acoplarse ni instanciar directamente las clases concretas. Esto garantiza que todas las naves respeten una abstracción común y sean entregadas con una configuración válida de componentes y recursos desde el primer momento.
- **Patrón State (Motor Warp - E1-02):**
  - Fue implementado por Uriel, quien además tuvo a cargo el sistema de Bitácora.
  - El patrón modela el ciclo de vida del Motor Warp mediante una secuencia estricta: Disponible → Preparando salto → En warp → Enfriamiento.
  - **Justificación:** Permite que cada estado encapsule su comportamiento y determine qué transiciones son válidas. Gracias a esto, el sistema puede rechazar y dejar constancia de transiciones inválidas (por ejemplo, intentar saltar directamente a "En warp" sin pasar por "Preparando salto") sin depender de un bloque centralizado de condicionales (if/switch).
- **Patrón Template Method (Ciclo de Misiones - E1-06):**
  - Estuvo a cargo de Agustin, responsable también del Asistente de Comando y los Informes de misión.
  - Se aplicó en las tres misiones simplificadas obligatorias: M-01 (Intercepción y asistencia), M-02 (Recolección) y M-03 (Retorno seguro).
  - **Justificación:** Se implementa para definir un algoritmo base inalterable con cuatro etapas obligatorias (preparar, ejecutar, evaluar, cerrar). Esto asegura la reutilización del ciclo general, permitiendo que cada misión específica redefina únicamente los pasos permitidos sin alterar el orden lógico del proceso.
- **Patrón Decorator (Liquidación de Haberes - E1-08):**

    - Fue desarrollado por Jeremías, junto con el modelo general de la Tripulación.
    - Se utiliza para calcular la remuneración mensual componiendo el sueldo base del cargo, el porcentaje de antigüedad, el subsidio por origen (Terrícola, Vulcano, Marciano) y conceptos extra como los consejos registrados.
    - **Justificación:** Su uso es obligatorio para evitar una jerarquía rígida y una "explosión de clases" que intente cubrir todas las combinaciones posibles de cargo, origen y antigüedad. Permite componer el haber dinámicamente en tiempo de ejecución (mediante clases como SubsidioPlanetaDecorator o AntiguedadDecorator) manteniendo claramente identificable el aporte económico de cada concepto.

Estructura de Pruebas y Demostración

Para probar el correcto funcionamiento del programa y verificar los requerimientos, el equipo estructuró una clase MainDemostracion ubicada fuera de los paquetes de dominio (para no acoplar el núcleo a la consola, preparándolo para la futura interfaz gráfica de la Entrega 2). El código cliente demuestra de forma reproducible los siguientes escenarios obligatorios:

- **Escenario A:** Creación de las tres naves mediante Factory, asignación de tripulación válida, ejecución exitosa de las misiones M-01, M-02 y M-03 con Template Method, y generación del informe y bitácora final.
- **Escenario B:** Intento de preparación de una misión en una nave sin recursos suficientes, verificando que el sistema rechace la operación sin alterar parcialmente el estado.
- **Escenario C:** Recorrido por la secuencia válida del Motor Warp (State) e intento forzado de una transición inválida para comprobar su rechazo.
- **Escenario D:** Demostración del cálculo de haberes mediante decoradores encadenados y comprobación de contratos invariantes (como el rechazo a antigüedades negativas).

Integridad y Registro de IA

El grupo cumplió con la normativa de integridad del TPG registrando el uso de herramientas de Inteligencia Artificial (Gemini y GPT-6) en el archivo docs/uso-ia.md. Se documentaron consultas relevantes que influyeron en el diseño, como la estructuración del patrón Decorator para que no rompa el encapsulamiento de la clase abstracta Tripulante, y la simplificación del método toString() de la clase InformeMision, detallando en cada caso la modificación realizada y la forma en que el equipo verificó los resultados. También se utilizó Claude, aportando ayuda para el patrón Template del ciclo general de las misiones y ejecuciones abstractas para los tipos de misión, la clase InformeMsion y AsistenteDeComandos para enlazar los métodos de las clases que lo une